



Kubernetes Services

Service

Since pods are ephemeral, meaning they're created and destroyed on a regular basis, we use Kubernetes services to keep track of changes on the pod. Services provide functionality like load balancing, service discovery between apps, and support for zero-downtime app deployments.

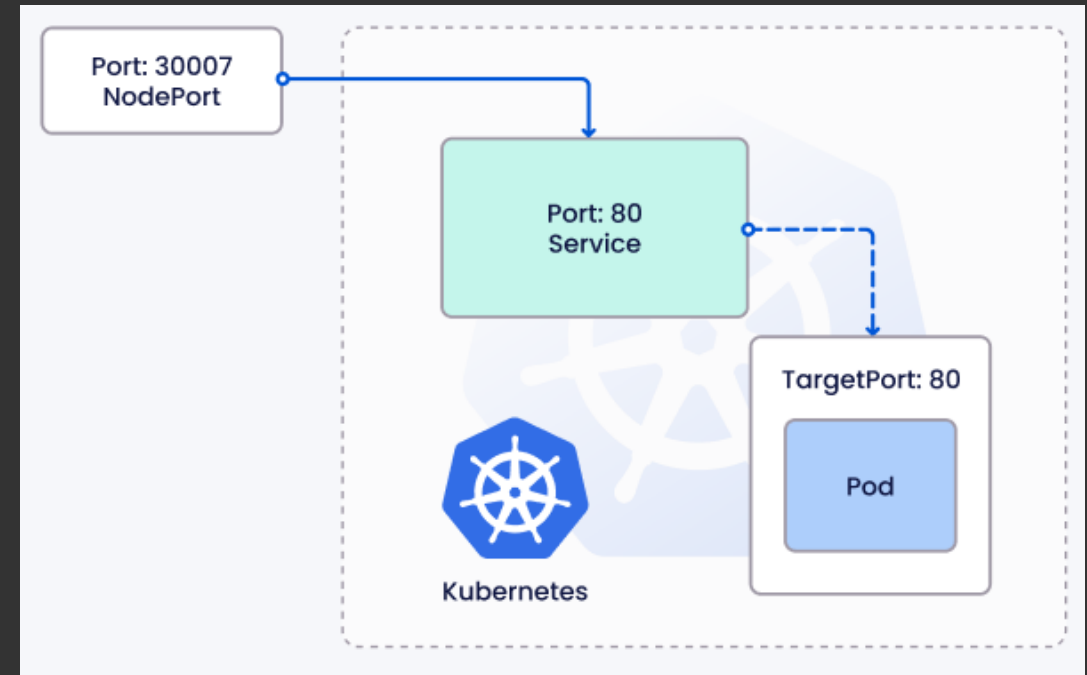
Kubernetes services expose a single endpoint for your application in the form of virtual IP. All the requests made to your service get redirected to appropriate pods, and thus service becomes a single endpoint for application communication.

Service Ports

TargetPort: The target port is the port where your pod is listening, and the service will send the request (targetPort: 80).

Port: Port exposes the Kubernetes service on the specified port in the cluster. Pods within the cluster can use this port to communicate with each port (port: 80).

NodePort: NodePort exposes the service externally to the cluster (NodePort 30008). NodePort can only be between 30000 and 32767



Service Types

ClusterIP: A ClusterIP creates a service inside the Kubernetes on a cluster-internal IP. ClusterIP is the default service type.

NodePort: The NodePort service, like ClusterIP, is assigned an IP address, but it also exposes the application on a port shared by all nodes in the cluster. Any request for the particular port has been redirected and forwarded to one or more specified nodes.

LoadBalancer: Every cloud provider (e.g., AWS, GCP, Azure, etc.) has its own load balancer service. Depending on which provider you're deploying your K8S application, the LoadBalancer Service will access the native implementation of that cloud service. In other words, the K8S engine will utilize the APIs of the provider's Load Balancer upon which it is deployed.

Manage Services

- `kubectl get svc` : (Lists all services in the current namespace)
- `kubectl get svc -A` : (Lists all services across all namespaces)
- `kubectl describe svc <service-name>` : (Shows detailed info about a service)
- `kubectl expose deployment <deployment-name> --port=<port> --target-port=<container-port> --type=<service_type>` : (creates a service and exposes a deployment to port)
- `kubectl apply -f <service-config.yaml>` : (Creates or updates a service from a YAML configuration file)
- `kubectl delete svc <service-name>` : (Deletes a specific service in the current namespace)
- `kubectl edit svc <service-name>` : (Edits an existing service configuration)
- `kubectl get nodes -o wide`: get nodes IP address
- `minikube service <service-name>`
- (Launches the service in a browser using Minikube's IP and assigned NodePort)

Service Configuration

deployment.yaml

```
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: myapp
5  spec:
6    selector:
7      matchLabels:
8        app: myapp
9      matchExpressions:
10       - {key: environment, operator: In, values: [dev]}
11       - {key: tier, operator: NotIn, values: [frontend,backend]}
12  template:
13    metadata:
14      labels:
15        app: myapp
16    spec:
17      containers:
18        - name: myapp
19          image: nginx
20          resources:
21            limits:
22              memory: "128Mi"
23              cpu: "500m"
24
```

service.yaml

```
1  apiVersion: apps/v1
2  kind: Service
3  metadata:
4    name: myapp
5  spec:
6    selector:
7      app: myapp
8    ports:
9      - port: 80
10      targetPort: 80
11
```

Service DNS Resolution

Kubernetes DNS, also known as kube-dns or CoreDNS, is a built-in service within Kubernetes that provides a reliable and scalable DNS-based mechanism for service discovery. It acts as a central lookup service, allowing services and pods to locate each other by name within the cluster.

Kubernetes DNS operates by creating a DNS record for each service and pod within the cluster. When a pod or service is created, Kubernetes automatically assigns it a DNS name in the form

<service-name>.<namespace>.svc.cluster.local

